



Command Design Pattern

Telmo Ferreira
Academia de Código

Command Design Pattern





```
public class Bathroom extends Room{  
    private Toilet toilet;  
    ...  
}
```

```
public class Bedroom extends Room{  
    private Bed bed;  
    ...  
}
```



```
public class Kitchen extends Room{  
    private Food food;  
    ...  
}
```



```
public class Light {  
  
    private boolean switchedOn;  
  
    public boolean isSwitchedOn() {  
        return switchedOn;  
    }  
  
    public void setSwitchedOn(boolean switchedOn) {  
        this.switchedOn = switchedOn;  
    }  
}
```



```
public class Room {  
  
    private Light light;  
  
    public Room() {  
        this.light = new Light();  
    }  
  
    public void switchLights() {  
        light.setSwitchedOn(!light.isSwitchedOn());  
    }  
}
```

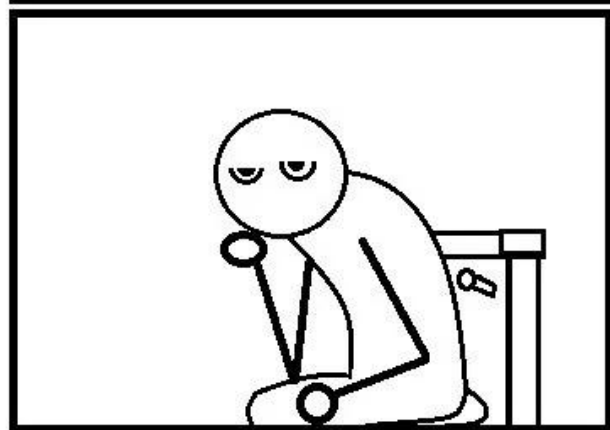
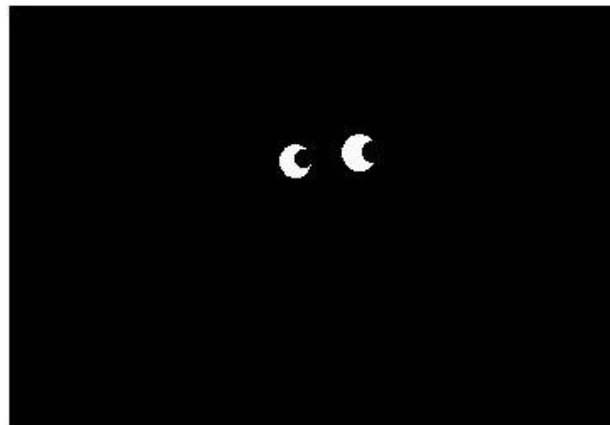
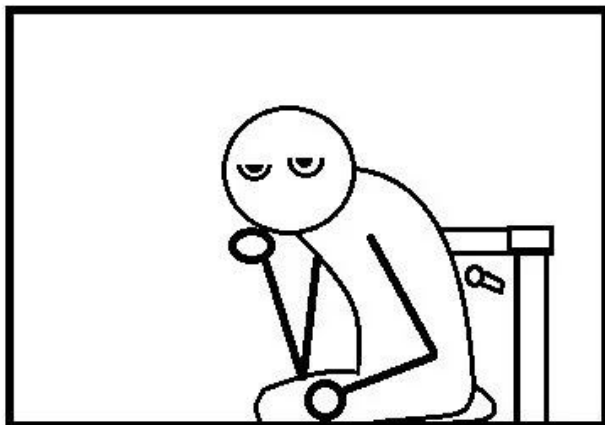


```
public class House {
    List<Room> rooms;

    public House() {
        rooms = new ArrayList<>();
    }

    public void addRoom(Room room) {
        rooms.add(room);
    }
}
```

```
public static void main(String[] args) {
    House house = new House();
    house.addRoom(new Bedroom());
    house.addRoom(new Kitchen());
    house.addRoom(new Bathroom());
    house.rooms.forEach(Room::switchLights);
}
```





```
public class FloorLamp {
    private Light light;

    public FloorLamp() {
        this.light = new Light();
    }
}
```

```
public class Bedroom extends Room{
    private FloorLamp floorLamp;
    private Bed bed;
    ...
}
```

```
public class Room {

    private Light light;

    public Room() {
        this.light = new Light();
    }

    public void switchLights() {
        light.setSwitchedOn(!light.isSwitchedOn());
    }
}
```



```
public class Light {
    private boolean switchedOn;

    public void switchLights() {
        switchedOn = !switchedOn;
    }
}
```

```
public class FloorLamp {
    private Light light;

    public FloorLamp() {
        this.light = new Light();
    }

    public void switchLights() {
        light.switchLights();
    }
}
```

```
public class Room {
    private Light light;

    public Room() {
        this.light = new Light();
    }

    public void switchLights() {
        light.switchLights();
    }
}
```



```
public class SwitchLightsCommand implements Command{
    private Light light;

    public SwitchLightsCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.switchLights();
    }
}
```



```
public static void main(String[] args) {
    Room bedroom = new Bedroom();
    bedroom.setCommand(new SwitchLightsCommand(new Light()));
    bedroom.executeCommand();
}
```



```
public interface Command {
    void execute();
}
```



```
public class Room {
    Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void executeCommand() {
        command.execute();
    }
}
```